

PROJECT 3 — END-TO-END DATA WAREHOUSE & ANALYTICS PIPELINE

Professional Project Journey & Technical Documentation

Project: End-to-End Data Warehouse using OLTP, Layered Data Processing and Analytics

Author: Shanmukha Vishnu Dasari

Program: B.Tech Artificial Intelligence & Data Science

Primary Career Track: Data Engineering

Status: Analytics and Power BI dashboard completed; documentation and repository packaging in progress

1. Executive Summary

This document records the complete development journey of Project 3, from the initial idea and architecture through OLTP design, ingestion, raw and staging layers, dimensional warehouse modeling, data-quality validation, analytical SQL, and Power BI reporting. It is written as a technical project record rather than as a simple README, so that the reasoning behind the implementation is preserved.

The project was initially planned as a multi-source data engineering system, but during implementation the focus shifted toward database design, layered data architecture, dimensional modeling, and understanding how a data warehouse is built end to end. The implemented core flow became:

Olist Source Data → Python Extraction → MySQL OLTP → Validation → PostgreSQL RAW → PostgreSQL STAGING → Data Warehouse → Analytical SQL → Power BI

Project scope clarification: The project was initially conceived with a multi-source requirement. As implementation progressed, the project deliberately prioritized Olist-based database design, OLTP modeling, layered PostgreSQL architecture, dimensional warehouse design, grain, surrogate keys, validation, analytics and BI. Additional API, JSON and Parquet sources remain future Version 2 extensions rather than completed components.

The implementation focused on understanding why each layer exists, how data grain is preserved, how source-system identities differ from warehouse surrogate keys, and how a dimensional model supports analytics.

2. Project Background and Starting Point

The project followed two earlier learning projects. Project 1 focused on employee data cleaning and analytics using Pandas, MySQL Connector and Tableau. Project 2 focused on building a modular mini ETL pipeline with configuration, logging, validation, transformation, writing, custom exceptions and object-oriented design.

Project 3 was created to move from isolated ETL/database exercises toward a more realistic end-to-end Data Engineering workflow.

- Project 1 established practical data cleaning and analytics fundamentals.
- Project 2 introduced structured ETL design and reusable Python components.
- Project 3 combined Olist source ingestion, OLTP, layered processing, dimensional modeling, analytics and BI.

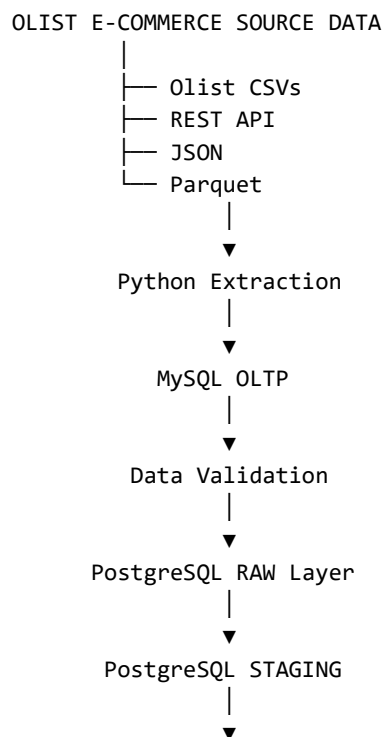
3. Business and Technical Objective

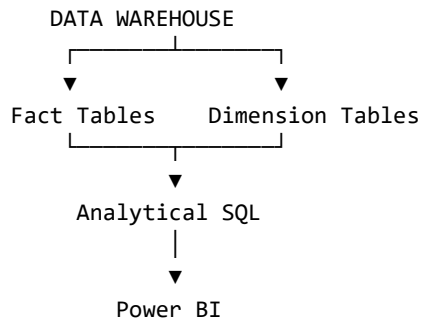
The objective was to build a realistic e-commerce data platform with strong database and warehouse architecture, starting from transactional OLTP design and progressing through raw, staging and dimensional warehouse layers to analytical SQL and Power BI.

Primary engineering goals

- Understand OLTP database design before building the warehouse.
- Design a strong database and warehouse architecture around the Olist transactional dataset.
- Separate raw, staging and warehouse responsibilities.
- Design fact tables from business processes and grain rather than copying OLTP tables.
- Use surrogate keys in dimensions while retaining source identifiers for lineage.
- Validate data at important boundaries.
- Produce reusable analytical SQL.
- Build a BI dashboard directly from the warehouse star schema.
- Document engineering decisions and current limitations honestly.

4. Initial Architecture





The design was kept intentionally layered. Each layer answers a different question: what did we receive, what did we clean, what does the business-ready model mean, and what can we analyze?

5. Development Journey — Day/Phase Record

The work evolved through a sequence of practical implementation phases. The labels below describe the actual learning and build progression. They avoid inventing calendar dates where an exact day was not recorded.

Phase 1 — Project Definition and Architecture

The first major task was to define the project as an end-to-end data warehouse project. A multi-source design was considered initially, but the implementation deliberately gave greater focus to database design and warehouse architecture.

- Defined the project as an end-to-end e-commerce data warehouse and analytics system.
- Selected Olist Brazilian e-commerce data as the primary transactional dataset.
- Initially planned JSON, API and Parquet inputs as future multi-source extensions, but did not implement them in the current version.
- Defined the conceptual flow from the Olist source data to BI.
- Identified MySQL as the OLTP layer and PostgreSQL as the analytical platform.
- Established the future possibility of S3/Snowflake without pretending those cloud components were already implemented.

Key learning: architecture should be designed around data flow and responsibilities, not around collecting technologies for a resume.

Phase 2 — OLTP Database Design

Before building the warehouse, the transactional side of the system was designed. The Olist dataset was examined to decide which entities were actually required.

- Identified customers, sellers, categories, products, orders, order items, payments and reviews.
- Added inventory and returns as future analytical entities.
- Applied normalization to separate related business entities.
- Identified primary keys and foreign-key relationships.
- Recognized parent-child relationships between orders and order items.
- Recognized cases where composite identity is required.
- Removed unnecessary fields from the final model when they did not support the project requirements.

A major learning point was that SQL syntax was not the hardest part. The harder problem was making the relationships between entities logically correct.

Phase 3 — MySQL OLTP Implementation

The Olist data was loaded into MySQL using Python and MySQL Connector, following dependency order so that foreign-key relationships could be respected.

```
customers → sellers → categories → products → orders → order_items → payments → reviews
```

- Implemented Python loaders for each logical entity.
- Used helper functions for integer and decimal conversion.
- Used ON DUPLICATE KEY UPDATE where appropriate.
- Mapped raw Portuguese product-category values to the normalized category identifiers.
- Verified that the resulting OLTP schema could store the transactional dataset.

Phase 4 — PostgreSQL Platform Setup

PostgreSQL was introduced as the analytical database platform.

```
CREATE DATABASE data_warehouse;

CREATE SCHEMA raw;
CREATE SCHEMA staging;
CREATE SCHEMA warehouse;
```

Three schemas were chosen to make layer boundaries explicit. RAW preserves received data, STAGING prepares standardized data, and WAREHOUSE contains the business-ready dimensional model.

Phase 5 — RAW Layer

The RAW layer was deliberately kept close to the source. Relational constraints were minimized so that incoming data could be preserved before transformation.

- Created RAW tables for customers, sellers, categories, products, orders, order_items, payments and reviews.
- Added extracted_at to track when a record entered the RAW layer.
- Built a metadata-driven MySQL-to-PostgreSQL loader.
- Used SELECT to control source columns.
- Used fetchall() to retrieve the selected rows.
- Used PostgreSQL executemany() for batch insertion.
- Used TRUNCATE plus reload for the current full-refresh implementation.

Important architectural decision: the PostgreSQL RAW schema is not a data lake. It is a raw/landing database layer. A future true data lake would use object storage such as S3.

Phase 6 — Data Validation Thinking

Validation was treated as a separate concern from loading. The reasoning was that valid OLTP data does not automatically guarantee valid staging data because transformations can introduce errors.

- Checked nulls in required fields.
- Checked duplicates at intended grains.

- Checked referential relationships.
- Checked row-count consistency.
- Checked value ranges and domains where applicable.
- Checked that transformations produced the intended output.
- Checked measure reconciliation between layers.

No separate data-quality folder was introduced at this stage. Checks were performed directly in PostgreSQL while the project was being built. Automation can be added later through orchestration.

Phase 7 — STAGING Layer

The STAGING layer was designed as the cleaning and standardization boundary.

- Used TRIM for text normalization.
- Used NULLIF where empty-string-to-NULL conversion was logically justified.
- Used DISTINCT ON with source timestamps where the latest record needed to be selected.
- Standardized columns before warehouse loading.
- Kept transformation logic in SQL rather than unnecessarily moving simple warehouse transformations into Python.
- Validated staging results before proceeding.

An important lesson was that not every column needs the same cleaning operation. Numeric columns do not need whitespace trimming, and NULLIF should only be used when an empty-value rule actually exists.

Phase 8 — Grain and Dimensional Modeling

The warehouse was not created by copying OLTP tables. Each fact table was designed from the business process and its grain.

Grain means exactly what one row represents. Grain was stated before writing fact-table SQL so that each transformation could be checked against the intended level of detail.

Fact 1 — fact_order_items

Grain: one row = one product line/item within one order.

- order_id acts as a degenerate dimension.
- order_item_id identifies the source line within the order.
- Customer, product and seller relationships use surrogate dimension keys.
- Purchase, approval, shipping-limit, carrier-delivery, customer-delivery and estimated-delivery dates use date keys.
- Price and freight_value are measures.
- Primary key: (order_id, order_item_id).

Fact 2 — fact_payments

Grain: one row = one payment record for an order.

- Primary key: (order_id, payment_sequence).
- Customer is linked through customer_key.
- Order purchase date is used because the payment source has no payment timestamp.
- payment_type and installments are descriptive/transactional attributes.

- payment_value is the measure.

Fact 3 — fact_reviews

Grain: one row = one review record for one order.

- Primary key: (review_id, order_id).
- Customer is linked through customer_key.
- Review creation and answer dates use date keys.
- review_score is retained as the analytical review measure/attribute.
- No product relationship was invented because the source did not provide a reliable product identifier for the review.

A key discovery was that review_id was not unique by itself in the staging data. The source contained the same review_id across different orders. The warehouse therefore used the composite key (review_id, order_id) instead of forcing an incorrect uniqueness assumption.

Phase 9 — Dimension Design

The final dimensions were customer, product, seller and date.

- dim_customer: customer_key, source customer identifiers, city and state.
- dim_product: product_key, product identifier, category information and product attributes.
- dim_seller: seller_key, seller identifier, city and state.
- dim_date: date_key, calendar attributes, week number and weekend indicator.

Natural/source identifiers were retained in dimensions for lineage and business identity. Surrogate keys were used for warehouse relationships.

Phase 10 — SCD Decision

Slowly Changing Dimensions were considered based on the actual source and business requirement rather than added mechanically.

- dim_customer: SCD Type 1 for the current static project.
- dim_product: SCD Type 1.
- dim_seller: SCD Type 1.
- dim_date: no historical SCD treatment required.

The Olist dataset is a static historical dataset. Therefore, implementing SCD Type 2 purely because locations could theoretically change would add complexity without representing a real historical-change requirement. A future dynamic source can justify SCD2 when historical states are required.

Phase 11 — Dimension Loading

- Generated dim_date from 2016-09-04 through 2020-04-09, producing 1,314 calendar rows.
- Loaded customers from staging and verified source-key uniqueness.
- Loaded products with a LEFT JOIN to categories so products without category matches were not silently removed.
- Observed 623 products with NULL category_id and verified that non-null category IDs matched the category table.
- Loaded sellers and verified uniqueness.

Phase 12 — Fact Loading and Warehouse Validation

Facts were loaded only after dimensions were available, allowing foreign-key resolution through surrogate keys.

- Loaded fact_order_items.
- Loaded fact_payments.
- Recreated fact_reviews using the correct composite primary key and loaded it successfully.
- Validated row counts.
- Validated grain-level duplicates.
- Validated dimension uniqueness.
- Validated foreign-key resolution.
- Validated date-key coverage.
- Validated required-column nulls.
- Reconciled important measures.

Phase 13 — Warehouse Reset and Reproducibility

A reset script was created so the warehouse could be rebuilt cleanly.

```
TRUNCATE TABLE
    warehouse.fact_reviews,
    warehouse.fact_payments,
    warehouse.fact_order_items,
    warehouse.dim_customer,
    warehouse.dim_product,
    warehouse.dim_seller,
    warehouse.dim_date
RESTART IDENTITY;
```

Facts are reset before dimensions because dimensions are referenced by fact foreign keys.

Phase 14 — Analytical SQL

The analytics layer was organized around business questions rather than around individual database tables.

```
analytics/
├── 01_business_kpis.sql
├── 02_sales_analysis.sql
├── 03_product_analysis.sql
├── 04_customer_analysis.sql
├── 05_geographical_analysis.sql
├── 06_payment_analysis.sql
├── 07_review_analysis.sql
└── 08_delivery_analysis.sql
```

- Business KPIs: revenue, freight, orders, items, AOV and review score.
- Sales: monthly revenue, month-over-month growth and cumulative revenue.
- Products: category revenue, top products and top products within category.
- Customers: top customers, rankings and one-time versus repeat behavior.
- Geography: revenue and order activity by state.
- Payments: payment methods and installment behavior.
- Reviews: review score distribution and averages.
- Delivery: on-time versus late performance and monthly delivery performance.

Advanced SQL was embedded into the relevant analytical files instead of creating a generic advanced-analysis folder.

Phase 15 — Power BI Integration

Power BI was selected for Project 3 to provide a different BI experience from Project 1, which used Tableau.

- Connected Power BI Desktop to the PostgreSQL warehouse.
- Resolved local PostgreSQL SSL/certificate behavior for the development connection.
- Loaded the warehouse star-schema tables rather than pre-aggregated analytics outputs.
- Verified relationships between dimensions and facts.
- Used dim_date with purchase date as the primary active date relationship for sales.
- Avoided creating a false product-to-review relationship.

Phase 16 — DAX and KPI Development

The dashboard was built from reusable DAX measures.

```
Total Revenue =  
SUM(fact_order_items[price])
```

```
Total Orders =  
DISTINCTCOUNT(fact_order_items[order_id])
```

```
Items Sold =  
COUNTROWS(fact_order_items)
```

```
Average Order Value =  
DIVIDE([Total Revenue], [Total Orders])
```

```
Average Review Score =  
AVERAGE(fact_reviews[review_score])
```

An on-time delivery measure was also implemented using actual customer delivery and estimated delivery date keys.

Phase 17 — Dashboard Development

The Power BI dashboard was built iteratively, visual by visual, with the warehouse star schema as the source.

- KPI cards: Total Revenue, Total Orders, Items Sold, Average Order Value and On-Time Delivery %.
- Monthly Revenue Trend.
- Revenue by Product Category.
- Top 10 Products by Revenue.
- Revenue by State.
- Payment Method Analysis.
- One-Time vs Repeat Customers.
- Review Score Distribution.
- Delivery Performance.
- Year, State and Category slicers.

The dashboard was refined manually in the Power BI interface. Layout alignment and visual sizing were treated as presentation work after the analytical model was verified.

6. Final Warehouse Model

```
warehouse
├── dim_customer
├── dim_product
├── dim_seller
├── dim_date
├── fact_order_items
├── fact_payments
└── fact_reviews
```

The final model follows a star-schema pattern: fact tables hold business events and measures, while dimensions provide descriptive context for slicing and analysis.

7. Key Engineering Decisions

Why RAW exists

RAW preserves what was received and provides a boundary for reprocessing and debugging. It isolates downstream transformations from the original ingestion result.

Why STAGING exists

STAGING is where source data becomes standardized and analytically safe. It separates cleaning logic from both ingestion and business-facing warehouse modeling.

Why the warehouse is not a copy of OLTP

OLTP is designed around operational transactions and source-system integrity. The warehouse is designed around analytical questions, business processes, grain, measures and dimensions.

Why multiple fact tables

Orders, payments and reviews represent different business processes and therefore have different grains. Combining them into one giant fact would risk duplication and incorrect measures.

Why surrogate keys

Surrogate keys provide stable warehouse relationships and support future dimensional-history strategies while source identifiers remain available for lineage.

Why not force SCD2

Historical dimension tracking should exist because the business needs historical states, not simply because an attribute could change in theory.

8. Important Problems and Lessons Learned

- Dataset design is harder than writing SQL.
- Grain must be defined before fact-table SQL.
- A source column that looks like an ID is not automatically unique.
- Composite keys are sometimes necessary to represent the true business/source grain.
- LEFT JOIN is important when unmatched dimension attributes must not remove valid facts/entities.

- A deadline such as `shipping_limit_date` must not be mislabeled as an actual event date.
- Payment records may need to use the order purchase date when no payment timestamp exists.
- Not every numeric dimension attribute is a measure; attributes describing an entity can belong in a dimension.
- OLTP constraints do not replace staging-level validation.
- A raw database schema is not the same thing as a data lake.
- Full-refresh loading is acceptable for a learning/local implementation but is different from true incremental processing.
- A dashboard should consume the warehouse model so that BI logic remains connected to the modeled business entities.

9. Current Limitations

- The current RAW-to-warehouse implementation uses full refresh rather than a production-grade incremental pipeline.
- Airflow orchestration is not yet implemented.
- Cloud storage/data lake deployment is not yet implemented.
- SCD Type 2 is planned as a future capability when a changing source and historical requirement justify it.
- Automated data-quality execution is not yet orchestrated.
- The current project uses a static historical Olist dataset, so real CDC behavior is not represented.

10. Planned Version 2 / Future Evolution

- Add Airflow for orchestration and scheduling.
- Containerize the pipeline with Docker.
- Introduce dynamic API ingestion.
- Move raw/object data toward AWS S3 as a real data lake layer.
- Introduce incremental loading instead of full refresh.
- Add automated data-quality checks.
- Implement SCD Type 2 where real historical changes exist.
- Consider Parquet-based lake storage and scalable processing.
- Evaluate Spark/Kafka/CDC only where project scale and use cases justify them.
- Add stronger deployment, monitoring and pipeline observability.

11. Project Structure

```
e2e-data-warehouse-pipeline/
├── architecture/
├── data/
├── etl/
├── oltp/
├── warehouse/
├── analytics/
├── dashboard/
├── documentation/
├── README.md
└── .gitignore
```

The repository is organized around pipeline responsibilities. Documentation should describe what is actually implemented and clearly separate current functionality from future plans.

12. Professional Project Outcome

The completed project demonstrates a progression from raw e-commerce source data to an analytical warehouse and BI layer. More importantly, it demonstrates the reasoning required to build such a system: entity modeling, normalization, source-to-target mapping, layered data architecture, grain definition, dimensional modeling, surrogate-key relationships, data-quality checks, analytical SQL and dashboard integration.

The project is therefore not just a collection of SQL scripts and visuals. It is a record of learning how a Data Engineer reasons about data as it moves through a system.

13. Skills Demonstrated

- SQL and relational database design
- MySQL OLTP design and implementation
- PostgreSQL schemas and warehouse modeling
- Python-based data extraction and loading
- Pandas and MySQL Connector experience
- Data cleaning and standardization
- Data validation and reconciliation
- Dimensional modeling and star schema
- Fact-table grain definition
- Surrogate keys and source-key lineage
- SCD strategy
- Analytical SQL including window functions and ranking
- Power BI and DAX
- Git/GitHub project organization
- Technical documentation and architecture reasoning

14. Final Learning Reflection

The most important change during this project was moving from thinking about individual SQL queries to thinking about the complete lifecycle of data. Each implementation decision was evaluated in terms of source identity, data quality, grain, lineage, transformation responsibility and analytical usefulness.

The project also showed why production-style data engineering cannot be reduced to a single ETL script. A maintainable pipeline needs clear layers, explicit contracts between stages, validation, reproducibility, meaningful modeling and an analytical interface.

The current implementation is intentionally a strong local foundation. The next stage is to make the same design more production-oriented through orchestration, containers, incremental processing, dynamic sources and cloud infrastructure.

15. Final Architecture Summary

```
graph TD
    A[SOURCE SYSTEMS] --> B[EXTRACTION]
    B --> C[MYSQL OLTP]
    C --> D[VALIDATION]
    D --> E[POSTGRESQL RAW]
    E --> F[POSTGRESQL STAGING]
    F --> G[DIMENSIONAL DATA WAREHOUSE]
    G --> H[ANALYTICAL SQL]
    H --> I[POWER BI]
```

Future production-oriented evolution:

```
graph TD
    A[Sources] --> B[AWS S3 / Data Lake]
    B --> C[ETL / Orchestration / Data Quality]
    C --> D[Warehouse / Lakehouse]
    D --> E[Power BI / Analytics]
```

16. Completion Checklist

- Project architecture defined
- OLTP ER design completed
- MySQL OLTP implemented
- PostgreSQL RAW layer implemented
- PostgreSQL STAGING layer implemented
- Staging validation performed
- Warehouse dimensions implemented
- Warehouse fact tables implemented
- Fact grain defined and validated
- Surrogate-key relationships implemented
- Warehouse validation completed
- Analytical SQL completed
- Power BI connected to warehouse
- DAX KPI measures implemented
- Dashboard visuals implemented
- Current limitations documented
- Future Version 2 direction defined

Document purpose: preserve the complete technical journey and decision-making record of Project 3 for portfolio, interview preparation, GitHub documentation and future project evolution.

End of document.